



UNIWERSYTET VIZJA

School of Computer Science & Technologies

Field of study: Computer Science

Maksim Kernazhytski

Index number: 46589

**Web Application: PocketBudget – Personal Budget Management
System**

Project documentation prepared under supervision of

Łukasz Krawczuk vel. Walczuk UV

Warsaw, 2026

Table of contents

1. Introduction
2. Project overview
3. Project objective
4. Target users
5. Functional requirements
6. Technologies used
7. System architecture
8. Database model
9. Max assistant
10. Charts and export
11. Security and user data
12. Testing
13. Installation instructions
14. User guide
15. Project management
16. Future improvements
17. Conclusion

1. Introduction

This document describes the design and implementation of the PocketBudget web application. The project was prepared as a university software development project. The application allows a user to track expenses, plan monthly budgets, set saving goals, view charts and receive simple local recommendations from the Max assistant.

The documentation explains the purpose of the application, the main functions, the technologies used, the system architecture, the database model, testing and installation steps.

2. Project overview

PocketBudget is a personal budget management system built as a multi-tier Django web application. The user can create an account and manage personal finance data in one place. The main idea is to give a clear overview of spending and help the user make better decisions about money.

The application is not only a basic CRUD project. It includes authentication, expense management, category management, monthly budgets, saving goals, chart generation, CSV/SVG export, admin statistics and the local assistant Max.

3. Project objective

The objective of the project is to build a working digital product that solves a practical problem. Many students and young users spend money every day but do not always see how much they spend per category or how close they are to their monthly budget.

PocketBudget helps the user collect this information and show it in a simple dashboard. The project also demonstrates the full lifecycle of a web

application: database design, backend logic, frontend pages, tests, documentation and version control.

4. Target users

The target users are people who need a simple and understandable personal finance tool. The project is especially useful for students and young adults because it focuses on daily expenses, monthly budget limits and saving goals.

- Students who want to track daily expenses.
- People who want a simple monthly budget overview.
- Users who prefer a clean dashboard instead of complicated finance tools.
- A teacher or evaluator who wants to test a complete Django application with authentication and database features.

5. Functional requirements

The main functional requirements are listed below. They describe what the user can do in the system and what the application must support.

Feature	Short description
Authentication	Users can register, log in and log out.
Expenses	Users can add, edit, delete and filter expenses by date and category.
Categories	Users can use default categories or create custom categories.
Budgets	Users can set an overall monthly budget and category budgets.
Saving goals	Users can track progress toward a money goal with a deadline.
Max assistant	Max gives short local tips based on spending data.
Charts	The system generates spending charts from saved data.
Export	Users can download charts as SVG and chart data as CSV.
Admin statistics	Admin users can see project activity and recent data.

6. Technologies used

The project was implemented with common and easy to explain technologies. The goal was to build a stable local web application that can be run on a university computer without external paid services.

Technology	Role in the project
Python	Main programming language.
Django	Backend framework, views, forms, ORM and authentication.
SQLite	Local database for project data.
Django ORM	Used to read and write database records.
HTML and CSS	User interface and visual style.
JavaScript	Small frontend actions and interface improvements.
SVG	Charts are generated as SVG images.
Git and GitHub	Version control and project repository.

7. System architecture

The project follows a layered Django architecture. Each layer has its own responsibility. This makes the project easier to understand, test and explain during the final presentation.

Layer	Implementation in this project
Presentation layer	Django templates, CSS and small JavaScript files.
Web / application layer	Django views, forms and URL routing.
Business layer	Max assistant logic, budget checks, goal progress and chart generation.
Data layer	Django models and ORM queries.
Infrastructure / admin	SQLite, migrations, virtual environment and Django Admin.

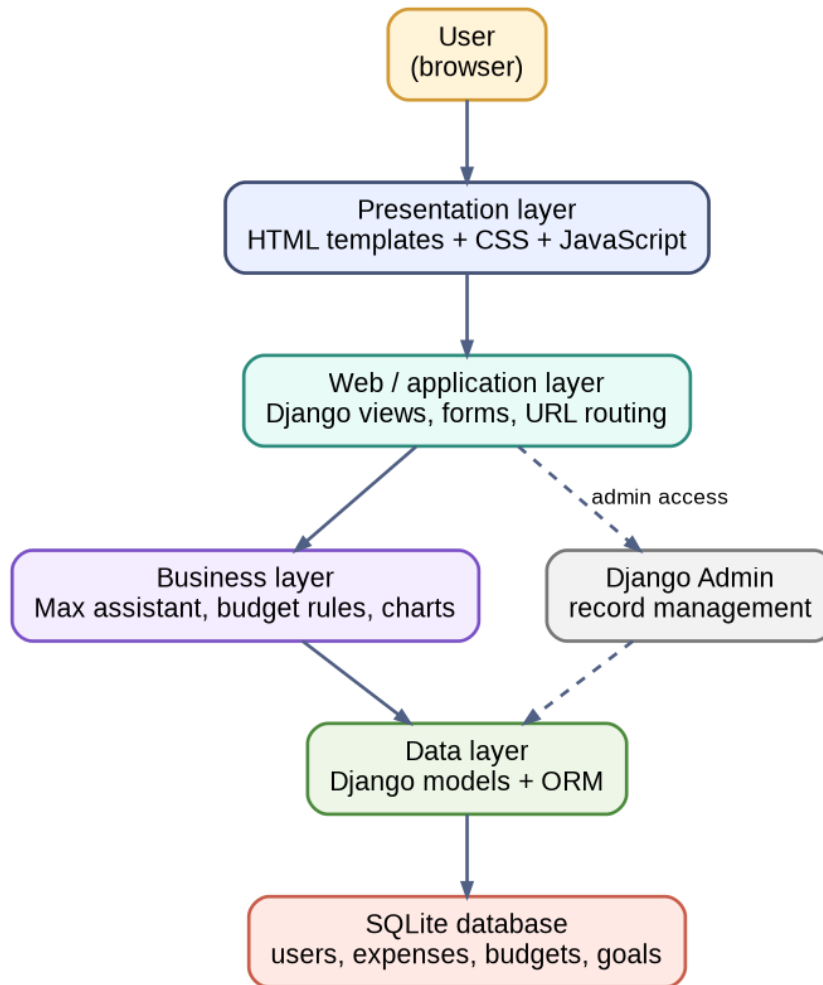


Figure 1. System architecture of PocketBudget.

8. Database model

The database model is simple but complete for the application requirements. It stores user data, expenses, budgets, saving goals, categories and assistant insights. Each user has separate data, so one user cannot see or modify the financial records of another user.

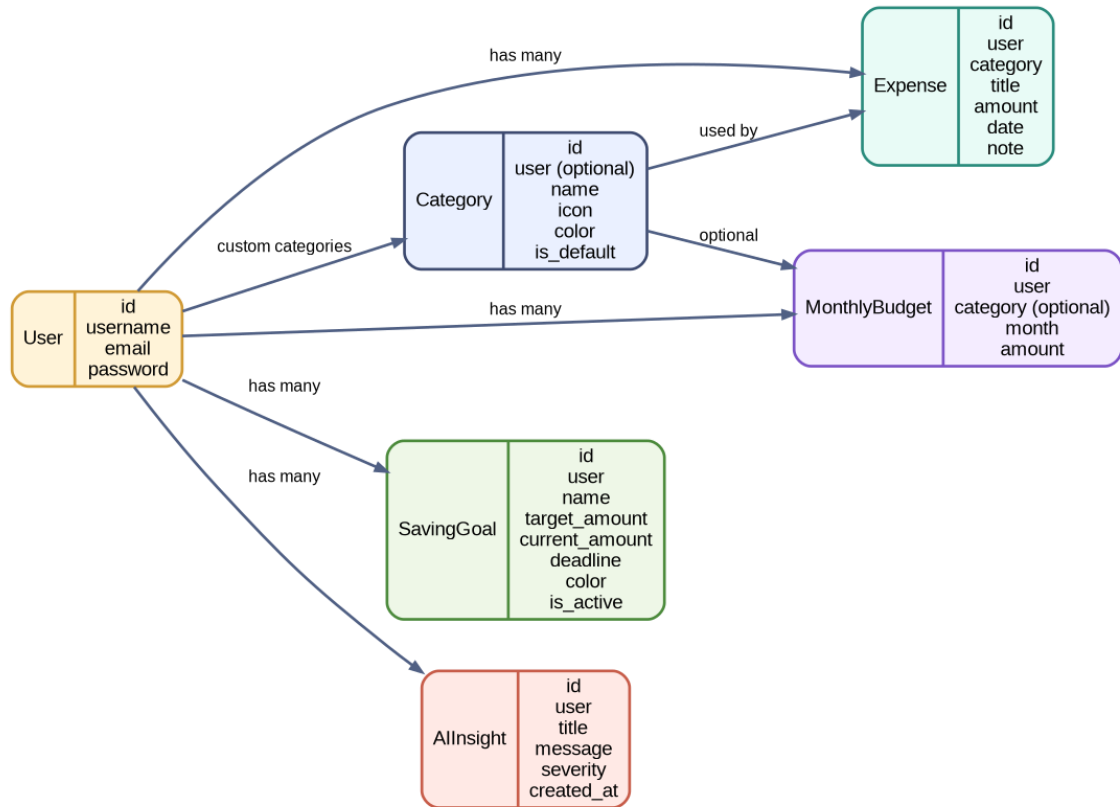


Figure 2. Database model used in the project.

Model	Purpose
Category	Stores default and user-created expense categories.
Expense	Stores user spending records.
MonthlyBudget	Stores overall and category budgets for a month.
SavingGoal	Stores user goals, progress and deadline.
AllInsight	Stores short tips generated by Max.
User	Standard Django user model used for authentication.

9. Max assistant

Max is the local budget assistant in the application. It is not connected to a paid external API. Instead, Max uses Python rules and data from the Django models. This makes the feature easy to explain and reliable for a university project.

- Max checks current month spending.
- Max compares expenses with the overall monthly budget.
- Max checks category budget usage.

- Max finds the biggest spending category.
- Max checks saving goal progress.
- Max comments on charts and spending patterns.

This makes the assistant part practical: Max reads saved data, applies local rules and gives useful output to the user.

10. Charts and export

The Charts page helps the user understand spending visually. The charts are generated from the user data stored in the database. The user can also download charts and export chart data.

Chart	What it shows
Spending by category	Shows which categories take the most money this month.
Daily spending trend	Shows spending for the last 14 days.
Budget usage	Compares monthly spending with the overall monthly budget.

Each chart can be downloaded as an SVG image. The chart data can also be exported as a CSV file, which can be opened in spreadsheet software.

11. Security and user data

The application uses Django authentication and standard Django security mechanisms. The goal is to keep user data separated and protect admin pages from regular users.

- Django authentication is used for login and registration.
- Users can only see their own expenses, budgets, goals and categories.
- Admin pages are protected and only staff users can open them.
- The project version uses a local SQLite database.

12. Testing

The project includes automated tests for the main functions. The tests check that important user actions work correctly. This is important because it proves that the application is not only visually finished, but also technically stable.

- Registration and login.
- Expense create, edit, delete and filters.
- Category create, edit and delete rules.
- Budget create, edit, delete and month filter.
- Goal create, edit, delete and deadline validation.
- Max tips page.
- Charts page and downloads.
- Admin statistics access.

The command used for testing is:

```
python manage.py test
```

The final version passed 27 automated tests successfully.

13. Installation instructions

To run the application locally, the user should open the folder where `manage.py` is located and run the following commands.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

After starting the server, the application can be opened in the browser at <http://127.0.0.1:8000/>.

14. User guide

The basic user flow is simple. A new user creates an account, adds financial information and then uses the dashboard, Max tips and charts to understand personal spending.

- Register a user account or log in.
- Create custom categories if needed.
- Add expenses with category, amount and date.
- Create an overall monthly budget.
- Create category budgets if needed.
- Add saving goals.
- Open Max Tips to see suggestions.
- Open Charts to view spending charts and export data.

15. Project management

The project can be managed with a simple board in Trello or GitHub Projects. A good board can have the columns To Do, In Progress, Testing and Done. This shows the project lifecycle and makes team work easier to organize.

Task group	Example tasks
Backend	Models, forms, views, authentication and CRUD logic.
Frontend	Templates, dashboard, CSS and page layout.
Data and assistant logic	Max rules, chart generation, export and tests.
Documentation	README, technical documentation and presentation.

16. Future improvements

The current version is complete for the university project, but the application can be extended in the future.

- Add a date range filter for reports.
- Add more chart types.

- Add PDF export for monthly reports.
- Add optional PostgreSQL support for production.
- Add profile settings for currency and theme.

17. Conclusion

PocketBudget is a working Django business application. It has user accounts, CRUD operations, database models, charts, export tools, admin statistics and a local assistant. The project is simple enough to understand, but it still shows the full flow of a multi-tier web application.

The application also includes automated tests and a GitHub repository, so the final version can be checked, installed and evaluated in a clear way.